

Picomatch	Tests passing		
1	2	3	4
8 Table of Contents Click to expand - Install - Usage - API - picomatch - test - matchBase - isMatch - parse - scan - compileRe	7 - Accurate matching - Using wildcards (*) and (?), globstars (**) for nested directories, advanced globbing with extglobs, braces, and POSIX brackets, and support for escaping special characters with \ or quotes. - Well tested - Thousands of unit tests See the library comparison libraries.	6 - Lightweight - No dependencies - Minimal - Tiny API surface. Main export is a function that takes a glob pattern and returns a matcher function. - Fast - Loads in about 2ms (that's several times faster than a single frame of a HD movie at 60fps) - Performant - Use the returned matcher function to speed up repeat matching (like when watching files)	5 Blazing fast and accurate glob matcher written in JavaScript. No dependencies and full support for standard and extended Bash glob features, including braces, extglobs, POSIX brackets, and regular expressions.
9 - Library Comparisons - Matching special characters as literals - Braces - Advanced globbing - Basic globbing - Globbing features - Options Examples - Scan Options - Picomatch options - Options - toRegex - makeRe	10 Install Install with npm: - Benchmarks - Philosophies - About - Author - License (TOC generated by verb using markdown-toc)	11 Usage The main export is a function that takes a glob pattern and an options object and returns a function for matching strings. <pre>const pm = require('picomatch'); const isMatch = pm('*.js');</pre>	12 <pre>console.log(isMatch('abcd')); //=> false console.log(isMatch('a.js')); //=> true console.log(isMatch('a.md')); //=> false console.log(isMatch('a/b.js')); //=> false</pre>
16 {Object}: Returns an object with matching info. <pre>const picomatch = require('picomatch'); picomatch.test(input, regex[, options]);</pre>	15 <pre>console.log(isMatch('a.a')); //=> false console.log(isMatch('a.b')); //=> true</pre> <pre>Test input with the given regex. This is used by the main picomatch() function to test the input string.</pre> Params - input {String}: String to test. - regex {RegExp} <pre>Test input with the given regex. This is used by the main picomatch() function to test the input string.</pre> Params - input {String}: String to test. - regex {RegExp}	14 <pre>const picomatch = picomatch('*.!(*)'); // picomatch(glob[, options]); require('picomatch');</pre> Example - returns {Function=: Returns a matcher function. - options {Object=: Returns a matcher function. - globs {String Array}: One or more glob patterns.	13 Creates a matcher function from one or more glob patterns. The returned function takes a string to match as its first argument, and returns true if the string is a match. The returned matcher function also takes a boolean as the second argument that, when true, returns an object with additional information. picomatch

[illegible]

Option	Type	Default value	Description	capture	boolean	undefined	Backslash escape and treat stars as (**). Return matched support method
				contains	boolean	undefined	
				33			
Option	Type	Default value	Description	cwd	string	process.cwd()	Allows match the given directory
				debug	boolean	undefined	Current directory
				dot	boolean	false	Debug expression an error
34							
Option	Type	Default value	Description	expandRange	function	undefined	Enable matching default ignored is explicit in the pattern Custom expand brace pattern such as
				35			
				36			
Option	Type	Default value	Description	fastpaths	boolean	true	The function receives values arguments must return to be used generally. It's recommended that relative strings in parentheses
				failglob	boolean	false	same as bash or found. no match throws
				37			
Option	Type	Default value	Description	format	function	undefined	Custom will be the node Regexp in the setting this behavior
				flags	string	undefined	Regex in the setting this behavior
				38			
Option	Type	Default value	Description	ignore	string	undefined	Ignore array\string that should be excluded
				literalBrackets	boolean	undefined	One or more POSIX pattern slashes removed. This is returned
				keepQuotes	boolean	false	POSIX pattern slashes removed. This is returned
39							
Option	Type	Default value	Description	matchBase	boolean	false	pattern escape only literal will be
				maxLength	boolean	65536	Alias for Limit the length string. thrown
				40			
Option	Type	Default value	Description	nobrace	boolean	false	Disable matching {a,b} would be literal
				nobacket	boolean	undefined	Disable with re bracket
				41			
Option	Type	Default value	Description	nocase	boolean	false	Make non case-insensitive Equivalent regex that the override flags
				nodupes	boolean	true	Deprecated noundup This option remove
				42			
Option	Type	Default value	Description	onIgnore	function	undefined	Function called to expand pattern treat the (like a regex) Disable
				onMatch	function	undefined	Function called to expand pattern treat the (like a regex) Disable
				43			
Option	Type	Default value	Description	noextglob	boolean	false	Disable extglob (b))
				noextglob	boolean	false	Disable extglob (b))
				44			
Option	Type	Default value	Description	noquantifiers	boolean	false	Disable quantifiers
				nonegate	boolean	false	Disable negation
				noglobstar	boolean	false	Disable globstar
45							
Option	Type	Default value	Description	posixSlashes	boolean	undefined	Convert slashes to backslashes. This does not affect the glob itself
				46			
				47			
Option	Type	Default value	Description	onResult	function	undefined	Function called to expand pattern treat the (like a regex) Disable
				48			
				49			

<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>prepend</td><td>boolean</td><td>undefined</td><td>String to the left of the regex to match. Use regex expressions + (instead of * and for following characters)</td></tr><tr><td>regex</td><td>boolean</td><td>false</td><td></td></tr><tr><td colspan="4">49</td></tr></table>	Option	Type	Default value	Description	prepend	boolean	undefined	String to the left of the regex to match. Use regex expressions + (instead of * and for following characters)	regex	boolean	false		49				<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>strictBrackets</td><td>boolean</td><td>undefined</td><td>Throw bracket parens imbalance error</td></tr><tr><td>strictSlashes</td><td>boolean</td><td>undefined</td><td>When the picomatch match slashes and]*</td></tr><tr><td colspan="4">50</td></tr></table>	Option	Type	Default value	Description	strictBrackets	boolean	undefined	Throw bracket parens imbalance error	strictSlashes	boolean	undefined	When the picomatch match slashes and]*	50				<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>unescape</td><td>boolean</td><td>undefined</td><td>Remove backslashes preceding characters glob pattern default are retained</td></tr><tr><td>unixify</td><td>boolean</td><td>undefined</td><td></td></tr><tr><td colspan="4">51</td></tr></table>	Option	Type	Default value	Description	unescape	boolean	undefined	Remove backslashes preceding characters glob pattern default are retained	unixify	boolean	undefined		51				<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td colspan="4">picomatch has automatic detection for regex positive and negative lookbehinds. If the pattern contains a negative lookbehind, you must be using Node.js >= 8.10 or else picomatch will throw an error.</td></tr><tr><td colspan="4">52</td></tr></table>	Option	Type	Default value	Description	picomatch has automatic detection for regex positive and negative lookbehinds. If the pattern contains a negative lookbehind, you must be using Node.js >= 8.10 or else picomatch will throw an error.				52																																																											
Option	Type	Default value	Description																																																																																																																				
prepend	boolean	undefined	String to the left of the regex to match. Use regex expressions + (instead of * and for following characters)																																																																																																																				
regex	boolean	false																																																																																																																					
49																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
strictBrackets	boolean	undefined	Throw bracket parens imbalance error																																																																																																																				
strictSlashes	boolean	undefined	When the picomatch match slashes and]*																																																																																																																				
50																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
unescape	boolean	undefined	Remove backslashes preceding characters glob pattern default are retained																																																																																																																				
unixify	boolean	undefined																																																																																																																					
51																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
picomatch has automatic detection for regex positive and negative lookbehinds. If the pattern contains a negative lookbehind, you must be using Node.js >= 8.10 or else picomatch will throw an error.																																																																																																																							
52																																																																																																																							
<table><tr><td colspan="4">53</td></tr><tr><td colspan="4">In addition to the main picomatch options, the following options may also be used with the .scan method.</td></tr><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>tokens</td><td>boolean</td><td>false</td><td>When true, the returned object will include an array of tokens (objects),</td></tr><tr><td colspan="4">54</td></tr></table>	53				In addition to the main picomatch options , the following options may also be used with the .scan method.				Option	Type	Default value	Description	tokens	boolean	false	When true, the returned object will include an array of tokens (objects),	54				<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>parts</td><td>boolean</td><td>false</td><td>When true, the returned object will include an array of strings representing each path "segment" in the scanned glob pattern. This is automatically enabled when options.tokens is true</td></tr><tr><td colspan="4">55</td></tr></table>	Option	Type	Default value	Description	parts	boolean	false	When true, the returned object will include an array of strings representing each path "segment" in the scanned glob pattern. This is automatically enabled when options.tokens is true	55				<table><tr><th>Option</th><th>Type</th><th>Default value</th><th>Description</th></tr><tr><td>Example</td><td></td><td></td><td></td></tr><tr><td colspan="4">56</td></tr></table>	Option	Type	Default value	Description	Example				56				<table><tr><td colspan="4">57</td></tr><tr><td colspan="4">The following example shows how to create a glob that matches a folder</td></tr><tr><td colspan="4"><pre>const fill = require('fill-range'); const regex = pm.makeRe('foo/{01..25}/bar', { expandRange(a, b) { return `(\${fill(a, b, { toRegex: true })})`; } }); console.log(regex);</pre></td></tr><tr><td colspan="4">60</td></tr></table>	57				The following example shows how to create a glob that matches a folder				<pre>const fill = require('fill-range'); const regex = pm.makeRe('foo/{01..25}/bar', { expandRange(a, b) { return `(\${fill(a, b, { toRegex: true })})`; } }); console.log(regex);</pre>				60																																																											
53																																																																																																																							
In addition to the main picomatch options , the following options may also be used with the .scan method.																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
tokens	boolean	false	When true, the returned object will include an array of tokens (objects),																																																																																																																				
54																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
parts	boolean	false	When true, the returned object will include an array of strings representing each path "segment" in the scanned glob pattern. This is automatically enabled when options.tokens is true																																																																																																																				
55																																																																																																																							
Option	Type	Default value	Description																																																																																																																				
Example																																																																																																																							
56																																																																																																																							
57																																																																																																																							
The following example shows how to create a glob that matches a folder																																																																																																																							
<pre>const fill = require('fill-range'); const regex = pm.makeRe('foo/{01..25}/bar', { expandRange(a, b) { return `(\${fill(a, b, { toRegex: true })})`; } }); console.log(regex);</pre>																																																																																																																							
60																																																																																																																							
<table><tr><td colspan="4">58</td></tr><tr><td colspan="4">Options Examples</td></tr><tr><td colspan="4">options.expandRange</td></tr><tr><td colspan="4">Type: function</td></tr><tr><td colspan="4">Default: undefined</td></tr><tr><td colspan="4">Custom function for expanding ranges in brace patterns. The fill-range library is ideal for this purpose, or you can use custom code to do whatever you need.</td></tr><tr><td colspan="4">Example</td></tr><tr><td colspan="4">62</td></tr></table>	58				Options Examples				options.expandRange				Type: function				Default: undefined				Custom function for expanding ranges in brace patterns. The fill-range library is ideal for this purpose, or you can use custom code to do whatever you need.				Example				62				<table><tr><td colspan="4">59</td></tr><tr><td colspan="4">options.format</td></tr><tr><td colspan="4">Type: function</td></tr><tr><td colspan="4">Default: undefined</td></tr><tr><td colspan="4">Custom function for formatting strings before they're matched.</td></tr><tr><td colspan="4">Example</td></tr><tr><td colspan="4">63</td></tr></table>	59				options.format				Type: function				Default: undefined				Custom function for formatting strings before they're matched.				Example				63				<table><tr><td colspan="4">64</td></tr><tr><td colspan="4">options.onMatch</td></tr><tr><td colspan="4">Type: function</td></tr><tr><td colspan="4">Default: undefined</td></tr><tr><td colspan="4">Custom function for formatting strings before they're matched.</td></tr><tr><td colspan="4">Example</td></tr><tr><td colspan="4">65</td></tr></table>	64				options.onMatch				Type: function				Default: undefined				Custom function for formatting strings before they're matched.				Example				65				<table><tr><td colspan="4">61</td></tr><tr><td colspan="4">options.onMatch</td></tr><tr><td colspan="4">Type: function</td></tr><tr><td colspan="4">Default: undefined</td></tr><tr><td colspan="4">Custom function for formatting strings before they're matched.</td></tr><tr><td colspan="4">Example</td></tr><tr><td colspan="4">66</td></tr></table>	61				options.onMatch				Type: function				Default: undefined				Custom function for formatting strings before they're matched.				Example				66			
58																																																																																																																							
Options Examples																																																																																																																							
options.expandRange																																																																																																																							
Type: function																																																																																																																							
Default: undefined																																																																																																																							
Custom function for expanding ranges in brace patterns. The fill-range library is ideal for this purpose, or you can use custom code to do whatever you need.																																																																																																																							
Example																																																																																																																							
62																																																																																																																							
59																																																																																																																							
options.format																																																																																																																							
Type: function																																																																																																																							
Default: undefined																																																																																																																							
Custom function for formatting strings before they're matched.																																																																																																																							
Example																																																																																																																							
63																																																																																																																							
64																																																																																																																							
options.onMatch																																																																																																																							
Type: function																																																																																																																							
Default: undefined																																																																																																																							
Custom function for formatting strings before they're matched.																																																																																																																							
Example																																																																																																																							
65																																																																																																																							
61																																																																																																																							
options.onMatch																																																																																																																							
Type: function																																																																																																																							
Default: undefined																																																																																																																							
Custom function for formatting strings before they're matched.																																																																																																																							
Example																																																																																																																							
66																																																																																																																							

[illegible]

Please read the contributing guide for advice on opening issues, pull requests, and coding standards. Running Tests Running and reviewing unit tests is a great way to get familiarized with a library and its API. You can install dependencies and run tests with the following command: <pre>npm install && npm test</pre> Building docs	<i>(This project's <code>readme.md</code> is generated by verb, please don't edit the <code>readme</code> directly. Any changes to the <code>readme</code> must be made in the .verb.md <code>readme</code> template.)</i> To generate the <code>readme</code>, run the following command: <pre>npm install -g verbose/verb#dev verb-generate-readme && verb</pre>	Author Jon Schlinkert • GitHub Profile • Twitter Profile • LinkedIn Profile License Copyright © 2017-present, Jon Schlinkert. Released under the MIT License.	
97	98	99	100
†01	£01	201	101
105	106	107	108
211	111	011	601